

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent Application Publication

20250285301

Kind Code

A1

Publication Date

September 11, 2025

Inventor(s)

Campagnolo Guizilini; Vitor et al.

SYSTEMS AND METHODS FOR PREDICTING A DEPTH MAP USING DIFFUSION-BASED MODELING

Abstract

System, methods, and other embodiments described herein relate to estimating a depth map from an image using a diffusion model that efficiently learns in optimal computational spaces using a learning model and trains with sparse data. In one embodiment, a method includes estimating a local vector for an image by combining random noise, an image embedding about the image, and a geometric embedding using a learning model, the local vector having depth information at a pixel-level for the image. The method also includes predicting a global vector by combining the image embedding and the geometric embedding at a scene-level by the learning model. The method also includes inferring a depth map of the image by combining the local vector and the global vector using a diffusion model.

Inventors: Campagnolo Guizilini; Vitor (Santa Clara, CA), Dave; Achal (San Francisco, CA), Tokmakov; Pavel (West Hollywood, CA), Ambrus; Rares A. (San Francisco, CA)

Applicant: Toyota Research Institute, Inc. (Los Altos, CA)

Family ID: 96949618

Assignee: Toyota Research Institute, Inc. (Los Altos, CA); Toyota Jidosha Kabushiki Kaisha (Toyota-shi Aichi-ken, JP)

Appl. No.: 18/776805

Filed: July 18, 2024

Related U.S. Application Data

us-provisional-application US 63561479 20240305

Publication Classification

Int. Cl.: G06T7/50 (20170101); G06T7/00 (20170101); G06T7/60 (20170101); G06T7/73 (20170101)

U.S. Cl.:

CPC G06T7/50 (20170101); G06T7/0002 (20130101); G06T7/60 (20130101); G06T7/73 (20170101); G06T2207/20081 (20130101)

Background/Summary

CROSS-REFERENCE TO RELATED APPLICATIONS [0001] This application claims the benefit of U.S. Provisional Application No. 63/561,479 filed on Mar. 5, 2024, which is herein incorporated by reference in its entirety.

TECHNICAL FIELD

[0002] The subject matter described herein relates, in general, to predicting a depth map from an image, and, more particularly, to predicting the depth map using a diffusion-based model and a geometric-based network through data conditioning.

BACKGROUND

[0003] Systems estimating depth for applications such as augmented reality (AR), medical imaging, automated driving, etc., encounter challenges. Applications mostly benefit from estimates having metric depth (i.e., meters) that capture the physical shape of the observed environment that is true-to-size and realistic. In this way, systems can reconstruct a three-dimensional environment that is scale-aware. Although recovering the metric depth using multi-view sources (e.g., stereo cameras) is feasible, systems using images from a lone camera (e.g., a monocular camera, a red-green-blue (RGB), etc.) have difficulties with accurately estimating metric depth.

[0004] In various implementations, systems use a learning model that estimates depth using data from a monocular camera and learned priors from training data. In this way, the systems reason over scale ambiguity and generate accurate predictions. However, the priors can lack expressiveness to generalize across diverse domains for the depth estimates. Furthermore, systems using a learning model to detect and learn the priors may demand training data that is diverse and scaled. As such, training the learning model with sparse data can lead to inaccurate and erroneous depth estimates during implementation. This is particularly challenging when the learning model is tasked with predicting metric depth. Therefore, systems having a learning model that estimates depth from data derived using a lone camera can be inhibited by training data for computations involving scaling and predicting metric depth.

SUMMARY

[0005] In one embodiment, example systems and methods relate to estimating a depth map from an image using a diffusion model that efficiently learns in optimal computational spaces using a learning model and trains with sparse data. In various implementations, systems constructing views having depth information and metric measures (e.g., meters) using a single image rather than stereo imaging (i.e., multiple cameras) face problems from scale ambiguity. A learning model can reduce scale ambiguity through leveraging increasingly vast labeled and unlabeled datasets. This generates geometric priors that allow generating consistent predictions across various scenarios but can have computation costs that are elevated from acquiring labeled data. A diffusion-based learning model can also reduce effects from scaling errors due to the generality and scalability of learned representations. However, diffusion-based models that estimate depth demand ground-truths that are dense for predictions similar to image generation through diffusion. These ground-truths can be depth labels that are unavailable. Thus, systems estimating depth using an image acquired through

a lone camera lack labeled data and priors for accurate and efficient computations through diffusion.

[0006] Therefore, in one embodiment, a prediction system infers a depth map from an image through learning physically-grounded priors using a geometric embedding and an image embedding that allows metric scaling derived with a learning model and a diffusion model. In particular, the prediction system estimates the depth map using the diffusion model that allows scalability for vast and diverse datasets lacking structure and training involving sparse data. This supports zero-shot transfer since the depth map is estimated for scenarios lacking and unobserved during training. The prediction system can also incorporate encodings for geometric positions that bridge domain gaps across datasets geometrically. In one approach, the prediction system conditions noise, an image embedding, and a geometric embedding (e.g., camera intrinsics, camera extrinsics, etc.) locally and globally with the learning model (e.g., a neural network) that improves performance for depth inferences across data ranges using the diffusion model. Here, the embeddings can be vectors having projected information and dimensionality information. The noise can be random noise inputted as an initial representation of the depth map. Furthermore, the image embedding can be structured into local tokens representing pixel-level information and global tokens that represent scene-level information that the diffusion model iteratively processes to complete the depth map. In this way, the prediction system efficiently and accurately estimates the depth map using diffusion at a pixel-level that allows metric scales regardless of having training data that is sparse and unstructured.

[0007] In one embodiment, a prediction system that estimates a depth map from an image using a diffusion model that efficiently learns in optimal computational spaces using a learning model and trains with sparse data is disclosed. The prediction system includes a memory storing instructions that, when executed by a processor, cause the processor to estimate a local vector for an image by combining random noise, an image embedding about the image, and a geometric embedding using a learning model, the local vector having depth information at a pixel-level for the image. The instructions also include instructions to predict a global vector by combining the image embedding and the geometric embedding at a scene-level by the learning model. The instructions also include instructions to infer a depth map of the image by combining the local vector and the global vector using a diffusion model.

[0008] In one embodiment, a non-transitory computer-readable medium for estimating a depth map from an image using a diffusion model that efficiently learns in optimal computational spaces using a learning model and trains with sparse data and including instructions that when executed by a processor cause the processor to perform one or more functions is disclosed. The instructions include instructions to estimate a local vector for an image by combining random noise, an image embedding about the image, and a geometric embedding using a learning model, the local vector having depth information at a pixel-level for the image. The instructions also include instructions to predict a global vector by combining the image embedding and the geometric embedding at a scene-level by the learning model. The instructions also include instructions to infer a depth map of the image by combining the local vector and the global vector using a diffusion model.

[0009] In one embodiment, a method for estimating a depth map from an image using a diffusion model that efficiently learns in optimal computational spaces using a learning model and trains with sparse data is disclosed. In one embodiment, the method includes estimating a local vector for an image by combining random noise, an image embedding about the image, and a geometric embedding using a learning model, the local vector having depth information at a pixel-level for the image. The method also includes predicting a global vector by combining the image embedding and the geometric embedding at a scene-level by the learning model. The method also includes inferring a depth map of the image by combining the local vector and the global vector using a diffusion model.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

[0010] The accompanying drawings, which are incorporated in and constitute a part of the specification, illustrate various systems, methods, and other embodiments of the disclosure. It will be appreciated that the illustrated element boundaries (e.g., boxes, groups of boxes, or other shapes) in the figures represent one embodiment of the boundaries. In some embodiments, one element may be designed as multiple elements or multiple elements may be designed as one element. In some embodiments, an element shown as an internal component of another element may be implemented as an external component and vice versa. Furthermore, elements may not be drawn to scale.

[0011] FIG. 1 illustrates one embodiment of a prediction system estimating a depth map using a diffusion model that efficiently learns within computational spaces using a learning model.

[0012] FIGS. 2A and 2B illustrate embodiments of training a recurrent interface network (RIN) for inferring the depth map through diffusion.

[0013] FIG. 3 illustrates one embodiment of the prediction system using a pipeline having a learning model and a diffusion model for estimating the depth map.

[0014] FIG. 4 illustrates an example of inferring the depth map through diffusion within a vehicle environment.

[0015] FIG. 5 illustrates one embodiment of a method that is associated with inferring a depth map of an image by combining a local vector and a global vector using the diffusion model.

DETAILED DESCRIPTION

[0016] Systems, methods, and other embodiments associated with inferring a depth map using a diffusion model that processes locally and globally conditioned information of image embeddings, geometric embeddings, and noise generated from a learning model are disclosed herein. In various implementations, systems using diffusion for estimating depth from monocular imaging involve fragmented architecture and ad-hoc adaptations that hamper performance and demand intensive training. For example, diffusion through latent auto-encoders specially trained with focused and less diverse datasets lead to inaccuracies and decreased sharpness. Furthermore, ground-truths that are dense can be unavailable for real-world datasets. Other systems that are trained to infill through interpolating missing values and step-unrolling that adds noise to the model output rather than the ground-truth also can exhibit constrained performance and capabilities. For instance, the systems can encounter complex computations inferring a depth map having metric scaling through diffusion without acquiring training data that is diverse. Thus, systems estimating a depth map using data from a lone camera face difficulties from limited training data and complexity involving specialized architecture.

[0017] Therefore, in one embodiment, a prediction system infers a depth map using a learning model and a diffusion architecture that is flexible and efficient to operate at a pixel-level while training with sparse and unstructured data. In one approach, the diffusion model is a recurrent interface network (RIN) that decouples core computation from input dimensionality when processing image data acquired from a lone camera (e.g., a monocular camera, red-green-blue (RGB) camera, etc.). The RIN can also remove system restrictions demanding image data formed as dense grids having spatial structure. In this way, the RIN increases computational efficiency over diffusion architectures such as a u-network (U-Net model) and exhibit domain-agnostic properties through allowing depth estimates across diverse datasets. In one embodiment, the RIN has self-attention layers that iteratively process image features having geometric positions in three-dimensions (3D) outputted by a learning model (e.g., a neural network) to condition the diffusion process both globally and locally. In this way, the prediction system generates a depth map at a pixel-level without specialized encoding (e.g., auto-encoders) while exhibiting real scaling. As

such, the prediction system infers a depth map using data from a lone camera in a metric scale and training with sparse data allowing zero-shot transfer, thereby exhibiting system robustness and diversity.

[0018] Moreover, in one embodiment, the prediction system estimates a local vector at a pixel-level for an image through combining random noise and a geometric embedding processed by the learning model. Here, the local vector can be a conditioned image embedding and the geometric embedding locally at pixel positions for the depth information. Embeddings can be vectors having projected information and dimensionality information formed to simplify parsing. For example, the image embedding has limited pixel data about objects within a shot. Meanwhile, the geometric embedding can be camera intrinsics (e.g., a focal length, a principal point, an aspect-ratio, etc.) defining the geometries for a camera and camera extrinsics having parameters that describe the camera relative to a reference frame. Furthermore, the learning model can predict a global vector by combining another image embedding about the image and the geometric embedding at a scene-level by the learning model. In this way, insights about a scene within an image are formed both locally and globally.

[0019] In various implementations, the prediction system forms tokens from the local and global vectors for simplifying processing by the diffusion model (e.g., a RIN). For instance, a token captures the contextual significance of an input element (e.g., a pixel) within a sequence that is derived through weighted aggregation from information associated with other elements. Regarding inferences, the diffusion model processes the random noise, local tokens, and global tokens to output noise representing a depth map. Diffusion of the local and global tokens continue with outputting noise until identifying a depth map that exhibits accurate features. In this way, the prediction system conditions the image locally and globally that allows the diffusion model to infer a depth map on a metric scale through combining information across varying datasets, thereby increasing system applicability and reliability.

[0020] Regarding training, the prediction system may acquire ground-truth depth about a monocular image having pixel data that is sparse and unstructured. The prediction system estimates a depth noise by a diffusion model (e.g., a RIN) combining injected noise and latent information with the ground-truth. Furthermore, the training involves calculating a loss as a difference between the depth noise and the injected noise and adapting the diffusion model with the loss. In one approach, the training reduces computation costs through removing an invalid subset of the pixel data by the diffusion model. Accordingly, the prediction system infers a depth map having a metric scale using data acquired from a camera through a diffusion model trained with sparse data and capable of diverse predictions.

[0021] It will be appreciated that for simplicity and clarity of illustration, where appropriate, reference numerals have been repeated among the different figures to indicate corresponding or analogous elements. In addition, the discussion outlines numerous specific details to provide a thorough understanding of the embodiments described herein. Those of skill in the art, however, will understand that the embodiments described herein may be practiced using various combinations of these elements.

[0022] With reference to FIG. 1, one embodiment of the prediction system **100** is further illustrated. The prediction system **100** can include a processor(s) **110** and a memory **120** that stores a depth module **130**. The memory **120** is a random-access memory (RAM), a read-only memory (ROM), a hard-disk drive, a flash memory, or other suitable memory for storing the depth module **130**. The depth module **130** is, for example, computer-readable instructions that when executed by the processor(s) **110** cause the processor(s) **110** to perform the various functions disclosed herein.

[0023] The prediction system **100** as illustrated in FIG. 1 is generally an abstracted form.

Furthermore, the depth module **130** generally includes instructions that function to control the processor(s) **110** to receive data inputs from a sensor (e.g., a camera). In one embodiment, the inputs represent observations of one or more objects in an environment proximate to a vehicle

and/or other aspects about a surroundings. As provided for herein, the depth module **130**, in one embodiment, acquires sensor data **160** that includes at least a camera image, such as an image captured by a monocular camera, a RGB camera, etc. In further arrangements, the depth module **130** acquires the sensor data **160** from further sensors such as a radar sensor, a light detection and ranging (LIDAR) sensor, etc., for locating and representing data about objects.

[0024] Moreover, in one embodiment, the prediction system **100** includes a data store **140**. In one embodiment, the data store **140** is a database. The database is, in one embodiment, an electronic data structure stored in the memory **120** or another data store and that is configured with routines that can be executed by the processor(s) **110** for analyzing stored data, providing stored data, organizing stored data, and so on. Thus, in one embodiment, the data store **140** stores data used by the depth module **130** in executing various functions. In one embodiment, the data store **140** includes the sensor data **160** along with, for example, metadata that characterize various aspects of the sensor data **160**. For example, the metadata can include location coordinates (e.g., longitude and latitude), relative map coordinates or tile identifiers, time/date stamps from when the separate sensor data **160** was generated, and so on. In one embodiment, the data store **140** further includes the embedding information **150** that includes vectors having projected information and dimensionality information. The embedding information **150** can include an image embedding derived from an image acquired using a lone camera (e.g., a monocular camera). For instance, the image embedding has pixel data that is sparse about objects within a view, a shot, etc. Meanwhile, the geometric embedding can be camera intrinsics and camera extrinsics. The camera intrinsics can include a focal length, a principal point, a pixel aspect-ratio, a skew, etc., defining the geometries for a camera. The camera extrinsics can include parameters that describe a position, an orientation, etc., of the camera relative to a reference frame.

[0025] Now turning to FIGS. 2A and 2B, an embodiment of training a RIN for inferring a depth map through diffusion are illustrated. Here, RIN operates as a diffusion model that efficiently trains using pixel data that is sparse and unstructured. For example, the RIN trains within a lower dimensionality space than an input space for an image. In this way, the RIN conducts expensive computations in lower dimensions and then project results back to the input space that reduces computational costs.

[0026] In FIG. 2A, a RIN block **210** learns a latent space $Z_{\text{sub.in}}$ by combining with input $X_{\text{sub.in}}$ (e.g., pixel data) through read and write operations iteratively until converging upon an output. For example, $Z_{\text{sub.in}}$ and $X_{\text{sub.in}}$ are tokens processed by self-attention layers and written back to output tokens $X_{\text{sub.out}}$. A self-attention layer can generate an output token capturing the contextual significance of an input element within a sequence that is derived through weighted aggregation from information associated with other elements. As previously explained, a token can capture the contextual significance of an input element within a sequence, such as a pixel. The contextual significance can be derived through weighted aggregation from information associated with other elements independent of input structure (e.g., spatial relationships), thereby increasing computational efficiency. Furthermore, the RIN block **210** can train efficiently using data subsets and dropping-out inputs that are invalid. In one approach, the RIN block **210** operates through a series of state transitions that are learned from a noise tensor ϵ to an image $x_{\text{sub.0}}$ associated with a data distribution. To learn a transition f , a forward function can be defined as:

$$[00001] \quad x_t = \sqrt{\gamma(t)}x_0 + \sqrt{1 - \gamma(t)}\epsilon, \quad \text{Equation (1)}$$

where $\epsilon \sim \text{custom-character}(0, 1)$, $t \sim \text{custom-character}(0, 1)$ and $\gamma(t)$ is a monotonically decreasing function. A learning network (e.g., a neural network) can learn to predict ϵ from $x_{\text{sub.t}}$ for transition step t with:

$$[00002] \quad \epsilon = f(x_t, t) = f(\sqrt{\gamma(t)}x_0 + \sqrt{1 - \gamma(t)}\epsilon, t). \quad \text{Equation (2)}$$

[0027] In one approach, Equation (2) involves sampling an image through a sequence of state transitions from $x_{\text{sub.1}} = \epsilon$ to $x_{\text{sub.0}}$ using small steps $x_{\text{sub.1}} \xrightarrow{\text{fwdarw.}} x_{\text{sub.1} - \Delta} \xrightarrow{\text{fwdarw.}} \dots$

.fwdarw.x.sub.0. This can involve controlling diffusion through “conditioning” that adds directions for computations. For instance, the condition is an additional variable y that represents one of a class label, a language caption, and camera parameters that control outputs.

[0028] In FIG. 2B, a RIN model **220** adapts to predict a depth map while training with pixel data that is sparse. As another improvement, the RIN model **220** infers the depth map using image data represented by tokens and the image data lacks structure, such as a dense grid. The RIN model **220** can separate computations of input tokens $X \in \text{custom-character.sup.N} \times D$ and latent tokens $Z \in \text{custom-character.sup.M} \times D$. Here, $X \in \text{custom-character.sup.N} \times D$ is derived from input data where N is dependent on input size and M is a fixed dimension. The RIN model **220** can execute a sequence of attention operations for training to infer a depth map. In one approach, the latent tokens Z attend to inputs X through a read operation followed by several self-attention computations on Z and the final write from latents to inputs forming a RIN block **210.sub.1**. Stacking the RIN block **210.sub.1** with the RIN block **210.sub.2** forms a deeper model that accurately and efficiently infers the depth map.

[0029] Regarding training details, the RIN model **220** can have a B number of RIN blocks where a block receives latent $Z_{\text{sub.b}}$ and input $X_{\text{sub.b}}$ tokens from a previous iteration and returns updated $Z_{\text{sub.b}+1}$ and $X_{\text{sub.b}+1}$ tokens. The RIN model **220** predicts injected noise $N(t) = \gamma(t)\epsilon$ as depth noise $\{\text{circumflex over (N)}\}(t)$ using a ground-truth depth associated with an input image. Here, $t \sim \text{custom-character}(0,1)$ is the diffusion timestep given noisy inputs $X_{\text{sub.0}} = D_{\text{sub.i}} + N(t)$. The ground-truth depth can be acquired from real data, synthetic data, etc., and the prediction system **100** can generate depth embeddings from ground-truth labels. Here, embeddings are vectors having rich information about projected data and dimensionality from inputs that are readily parsable, thereby reducing computational costs.

[0030] The computations by the RIN model **220** can involve self-attention among the latent tokens Z that is independent of the input size N . As such, the RIN model **220** can train the transition function directly in the pixel space even for high-resolution and unstructured input X tokens within a low-dimensionality space, thereby reducing overall computational costs. As previously explained, the RIN model **220** can train through calculating a training loss **230** as a difference between the depth noise $\{\text{circumflex over (N)}\}(t)$ and the injected noise $N(t)$. This can involve adapting parameters, weights, etc., for attention networks of the RIN block **210.sub.1** with the RIN block **210.sub.2** until converging upon a threshold for losses. Accordingly, the RIN model **220** trains to infer a map depth through diffusion and tokens at a pixel-level using pixel data that is sparse, thereby improving efficiency and robustness.

[0031] Now turning to FIG. 3, one embodiment of the prediction system **100** using a pipeline **300** having a learning model and a diffusion model for estimating a depth map is illustrated. Here, the prediction system **100** and/or the depth module **130** includes instructions that cause the processor **110** to estimate a local vector for an image by combining random noise and a geometric embedding using a learning model (e.g., a neural network) where the local vector has depth information at a pixel-level. The prediction system **100** predicts a global vector by combining an image embedding about the image and the geometric embedding at a scene-level by the learning model. In one approach, the depth module **130** infers a depth map of the image by combining the local vector and the global vector using a diffusion model. Although FIG. 3 references multiple embeddings, the prediction system **100** can infer a depth map from an image embedding representing an image and a geometric embedding representing camera parameters and variables. Furthermore, the prediction system **100** and the pipeline **300** can be interchangeable for descriptions about FIG. 3.

[0032] In FIG. 3, the RIN block **210** benefits from geometric information derived from the geometric embeddings as a source of priors using input **310**. Here, the priors can be assumptions, constraints, etc., imposed on network parameters that influence learning and guide solutions that align with a prior knowledge, expectations, etc. In one approach, a geometric RIN (GRIN) receives the input **310** as a depth map $D_{\text{sub.i}} \in \text{custom-character.sup.H} \times W$ that is sparse and the input

310 has pixel-wise $d_{sub}.ijk$ distances to a camera (e.g., a monocular camera, a RGB camera, etc.). As an improvement, the distances can be metric values representing physical distances that range between $d_{sub}.s$ and $d_{sub}.f$ for $j \in [0, H]$ and $k \in [0, W]$. Furthermore, depth can be represented as Euclidean depth having a distance along a viewing ray $r_{sub}.ijk$. In this way, the RIN block **210** allows parameterizations for large-scale views through utilizing viewing rays that are individualized.

[0033] Moreover, $d_{sub}.ijk$ can be missing and replaced with invalid values (e.g., zero) since $D_{sub}.i$ is assumed to be sparse. As such, the pipeline **300** conditions the input **310** for diffusion on image data $I_{sub}.i \in \text{custom-character.sup.H} \times \text{W} \times 3$ acquired from a monocular camera, a RGB camera, etc., and corresponding camera intrinsics $K_{sub}.i$ and extrinsics $T_{sub}.i$. As previously explained, conditioning can involve controlling computations through adding parameters, constraints, etc. The camera intrinsics $K_{sub}.i$ can include a focal length, a principal point, a pixel aspect-ratio, a skew, etc., defining camera geometries. The camera extrinsics $T_{sub}.i$ can include parameters that describe a position, an orientation, etc., of the camera relative to a reference frame. The prediction system **100** can transform 3D points into a coordinate system of the camera using this camera information.

[0034] In the pipeline **300**, the prediction system **100** can utilize an image and camera geometry formed within embeddings as input modalities that condition depth predictions. Here, the camera information conditioning diffusion allows inferring the depth map at a pixel-level rather than globally, thereby improving accuracy. The pipeline **300** can generate image embeddings **310.sub.1** and **310.sub.2** using an encoder $\text{custom-character.sub.}\theta$ for the input **310** having the input image $I_{sub}.i$. The encoding involves computing $F_{sub}.i = \text{custom-character.sub.}\theta(I_{sub}.i)$ having learnable parameters θ . In one approach, the pipeline **300** forms tokens from the image $I_{sub}.i$ using a single convolutional layer $\text{custom-character.sub.}\theta.\text{sup.loc}$ with kernel size $K \times K$ and $C_{sub}.l$ output channel dimensions. An output can be a flattened $F_{sub}.i.\text{sup.loc} \in \text{custom-character.sub.}\theta.\text{sup.HW} \times C_{sub}.l$ feature map containing patch-wise visual information $f_{sub}.ijk$ for a pixel $p_{sub}.ijk = (u, v)_{sub}.jk$ within the image $I_{sub}.i$.

[0035] The pipeline **300** can also generate geometric embeddings **3103** using information from the camera used to obtain the image $I_{sub}.i$. For example, the pixel $p_{sub}.ijk$ from image the $I_{sub}.i$ is parameterized from a camera center $t_{sub}.i = [x_{sub}.i, y_{sub}.i, z_{sub}.i]$ and viewing ray $r_{sub}.ijk = (K_{sub}.i R_{sub}.i)_{sub}.sup.-1[u_{sub}.jk, v_{sub}.jk, 1]_{sub}.sup.T$ assuming that $K_{sub}.i$ is a 3×3 matrix and $T_{sub}.i$ a 4×4 matrix. Increasing expressiveness can involve Fourier encoding values with a mapping of:

[00003]

$$\text{Math.}(x) \mapsto [x, \sin(f_1 x), \cos(f_1 x), \text{Math.}, \sin(f_N x), \cos(f_N x)]^T. \quad \text{Equation (3)}$$

Here, N can the number of encoding frequencies equally spaced between

[00004] $[1, \frac{1}{2}]$.

Assuming $N_{sub}.o$ frequencies for camera centers and $N_{sub}.r$ for viewing rays, the geometric embeddings **310.sub.3** can exhibit dimensionality

$$D = 2(3(N_{sub}.o + 1) + 3(N_{sub}.r + 1)) = 6(N_{sub}.o + N_{sub}.r + 2).$$

[0036] The pipeline **300** can utilize the geometric embeddings **310.sub.3** within $g_{sub}.ijk = \text{custom-character}(t_{sub}.i, r_{sub}.ijk) = \varepsilon(t_{sub}.i) \oplus \varepsilon(r_{sub}.ijk)$ to imbue visual information with geometric awareness. As such, the pipeline **300** derives features capable of reasoning over 3D properties such as physical shape and scale. Accordingly, the prediction system **100** through the pipeline **300** can implicitly infer multi-view geometry and zero-shot transfer (e.g., scenarios lacking training) of metric depth across atypical datasets and camera geometries that are diverse using image data derived from a lone camera (e.g., a monocular camera, a RGB camera, etc.).

[0037] In various implementations, the GRIN operates at a pixel-level without latent auto-encoders and the GRIN uses defined tokenizers that simplify computations. As further explained below, the

prediction system **100** uses depth embeddings from a ground truth about the input image during training since inferring depth maps can involve range intervals that are vast. In one approach, the prediction system **100** can independently verify a log-scale parameterization using projection and unprojection functions mapping $d_{\text{sub.ijk}}$ to and from log-space $\{\text{circumflex over (d)}\}_{\text{sub.ijk}}$ as:

$$[00005] \hat{d}_{ijk} = \log_b((b-1)^{\frac{d_{ijk}-d_s}{d_f-d_s}} + 1), d_{ijk} = \frac{b^{\hat{d}_{ijk}}-1}{b-1}(d_f-d_s) + d_s. \quad \text{Equation (4)}$$

Here, b is the logarithm base determining compression of distances at different ranges. In this way, the prediction system **100** through training makes shorter ranges robust to residual noise from the diffusion process while maintaining performance at longer ranges where the residual noise is less impactful.

[0038] Regarding conditioning, local conditioning **330** prepares the image embeddings **310.sub.1** and the geometric embeddings **310.sub.3** for diffusion through conditioning. The pipeline **300** can improve depth estimates with local conditioning through adding depth information that is unavailable at certain pixels when the image embedding **310.sub.1** has sparse and unstructured image data. In one approach, the local conditioning **330** concatenates corresponding tokens for the embeddings denoted by the symbol $\oplus$. Here, the pipeline **300** implements geometric embeddings **310.sub.3** as $g_{\text{sub.ijk}}$ describing a pixel in a 3D reference frame rather than a two dimensional (2D) location within the image $I_{\text{sub.i}}$. In this way, the diffusion focuses upon localized predictions for a depth map and promotes disambiguation between camera geometries (e.g., focal length, resolution, or distortion). In another approach, the conditioned vector is defined as $\{\text{circumflex over (d)}\}_{\text{sub.ijk}} = d_{\text{sub.ijk}} \oplus f_{\text{sub.ijk.sup.loc}} \oplus g_{\text{sub.ijk}}$ for a one-dimensional prediction $d_{\text{sub.ijk}} \in \text{custom-character}$ that is projected onto a V -dimensional vector $v_{\text{sub.ijk}}$ using a linear layer $\text{custom-character.sub.1} + c_{\text{sub.l.sub.}} + D_{\text{fwdarw.V.sup.loc}}$. Accordingly, the local conditioning **330** outputs conditioned vectors for possible HW predictions estimated during diffusion as $V_{\text{sup.loc}} \in \text{custom-character.sup.HW} \times V$ and represented with $\text{custom-character.sup.loc}$.

[0039] Regarding the global conditioning **340**, image embeddings **310.sub.2** are generated using a convolutional encoder $\text{custom-character.sub.}\theta_{\text{sup.glob}}$ resulting in multi-scale feature maps $F_{\text{sub.i.sup.glob}} = [F_{\text{sub.i.sup.0}}, F_{\text{sub.i.sup.1}}, \dots, F_{\text{sub.i.sup.S}}]$ at S increasingly lower resolutions, thereby differing from the image embeddings **310.sub.1**. To combine multi-scale information, lower-resolution feature maps are upsampled, concatenated, and flattened for generating $\{\text{circumflex over (F)}\}_{\text{sub.i.sup.glob}} \in$

$$[00006] \mathbb{R}^{\frac{HW}{d^2} \times C_g}.$$

Here, d is the downsampling factor of the highest encoded resolution and $C_{\text{sub.g}}$ is the concatenated channel-wise dimension. In one approach, the image embeddings **310.sub.2** contain image data that is multi-resolution, multi-scale, and scene-level yet dense. Although lacking pixel-level prediction for depth like the image embeddings **310.sub.1** about scene objects, the image embeddings **310.sub.2** can promote global consistency independent of depth during diffusion.

[0040] Moreover, the global conditioning **340** can incorporate spatial structure through combining image $\{\text{circumflex over (F)}\}_{\text{sub.i.sup.glob}}$ and geometric embeddings $G_{\text{sub.i}}$. Here, the pipeline **300** can generate the geometric embeddings **310.sub.3** from a camera resized to match resolution of the image embeddings **310.sub.2**. The pipeline **300** may concatenate embeddings projected onto V -dimensional vectors using a linear layer custom-character

$\text{sub.C.sub.g.sub.} + D_{\text{fwdarw.V.sup.glob}}$. The collection of M vectors used to globally condition the diffusion process can be given by $V_{\text{sup.glob}} \in \text{custom-character.sup.M} \times V$ and represented with $\text{custom-character.sup.glob}$.

[0041] Upon completing conditioning, the RIN block **210** receives the input tokens $X = V_{\text{sup.loc}} \oplus V_{\text{sup.glob}} \in \text{custom-character.sup.(N+M)} \times V$ and random noise $N_{\text{sub.t}}$ **320** (e.g., Gaussian noise) representing an initial depth map for the image $I_{\text{sub.i}}$. Here, $V_{\text{sup.loc}}$ can be

concatenated with $V_{sup.glob}$ as grouped tokens **350** that the RIN block **210** processes to predict noise across possible local tokens and output $\{\circlearrowleft(N)\}_{sub.t}$ as a depth map. In one approach, the prediction system **100** infers the depth map having the grouped tokens **350** representing pixels excluding information about spatial structure for objects within the image $I_{sub.i}$. The grouped tokens **350** can be in a series having a group of local tokens and a group of global tokens for simplifying computations. Furthermore, the RIN block **210** can run several times for predicting and removing noise using multiple sub-RIN blocks similar to that illustrated in FIG. 2B. Over time, the depth map $\{\circlearrowleft(N)\}_{sub.t}$ develops as an accurate representation of depth at a pixel-level through diffusion computations that are efficient even with sparse and unstructured pixel data within the image $I_{sub.i}$.

[0042] Now discussing training within the context of the pipeline **300**, the prediction system **100** can incorporate the geometric embeddings $310_{sub.3}$ as the spatial structure for conditioning inputs associated with tokens. Once incorporated, a token can be treated independently for structure. This property can allow the RIN block **210** to ingest training data that is sparse through simply discarding invalid information and operating on valid tokens that are remaining. In this way, the RIN block **210** can promote pixel-level training and inference that is efficient when combined with computational bottlenecks provided by latent tokens.

[0043] Moreover, valid and conditioned vectors can be $\{\circlearrowleft(V)\}_{sup.loc} \in \text{custom-character}_{sup.N \times V}$ assuming a ground-truth in a 2D grid $D_{sub.i} \in \text{custom-character}_{sup.H \times W}$ with $N < HW$ valid pixels. A depth embedding that is valid can be conditioned using an original 2D location defining a viewing ray as well as a corresponding image patch. Furthermore, global conditioning during training can mitigate decreasing information available for conditioning from valid image pixels without depth labels being discarded. For example, few remaining pixels can be available for conditioning with sparse depth labels ($N \ll HW$) that destroy scene structure. The sky is an example of a structure having few depth labels that are valid during supervised training. As such, global conditioning can train with unstructured sparse data while still maintaining scene-level information that is dense.

[0044] In one embodiment, a $\{\circlearrowleft(V)\}_{sup.loc}$ matrix exhibits a varying length N during training as pixels with missing depth information are discarded (i.e., $d_{sub.ijk}=0$). The pipeline **300** can improve iteration speed through randomly discarding a percentage of local vectors that are valid and supervising a subset of L pixels, such as through data dropout. This can lead to faster cross-attention with latent tokens from the RIN block **210**. Similarly, discarding information from global vectors can improve iteration speeds and overall performance. As such, resulting input tokens can be of dimensionality $\{\circlearrowleft(X)\} \in \text{custom-character}_{sup.(L+G) \times V}$. For depth estimation using data from a lone camera (e.g., a monocular camera, a RGB camera, etc.), the pipeline **300** conditions one image $I_{sub.i}$ globally resulting in $V_{sup.glob}=V_{sub.i.sup.glob}$ compared with stereo cameras that assumes two images $I_{sub.i}$ and $I_{sub.j}$ resulting in $V_{sup.glob}=V_{sub.i.sup.glob} \oplus V_{sub.j.sup.glob}$, thereby increasing efficiency.

[0045] A training objective of the pipeline **300** can involve minimizing a L2 loss. Here, the L2 loss can be a squared loss, a mean squared error (MSE) loss, etc., that measures the average squared difference between the actual and predicted values. In one approach, the pipeline **300** trains in a log-depth scale such that $\text{custom-character}(t)=(\epsilon \gamma(t)-\{\tilde{\circlearrowleft(\epsilon)}\})_{sup.2}$. In this equation, $\tilde{\text{custom-character}}(0)$ can be injected noise at timestep $t \sim \text{custom-character}(0,1)$ and $\{\tilde{\circlearrowleft(\epsilon)}\}$ is the predicted noise by the RIN block **210** at a timestep. Therefore, the pipeline **300** trains to efficiently infer a depth map using data from a lone camera in a metric scale and sparse and unstructured data through conditional diffusion, thereby promoting system reliability and robustness.

[0046] Turning to FIG. 4, one embodiment of inferring a depth map through diffusion by the prediction system **100** within a vehicle environment **410** is illustrated. Here, a vehicle **420** is merging onto a road having a median **430** and the pickup truck **440** to the left. The vehicle **420** uses

one of a monocular camera, a RGB camera, etc., to capture the vehicle environment **410** and stores resulting image data in the sensor data **160**. In one approach, the prediction system **100** runs on the vehicle **420** and estimates a local vector for the image data through combining random noise and a geometric embedding using a learning model. The local vector can have depth information at a pixel-level for the image data. Furthermore, the prediction system **100** predicts a global vector by combining an image embedding about the image data and the geometric embedding at a scene-level by the learning model. For inferring the depth map, the depth module **130** may combine the local vector and the global vector using a diffusion model. In this way, the vehicle **420** efficiently derives a depth map that is accurate and reliable for executing downstream tasks such as automated driving, object detection, etc.

[0047] Regarding FIG. 5, a flowchart of a method **500** that is associated with inferring a depth map of an image by combining a local vector and a global vector about the image using a diffusion model is illustrated. The method **500** will be discussed from the perspective of the prediction system **100** of FIG. 1. While the method **500** is discussed in combination with the prediction system **100**, it should be appreciated that the method **500** is not limited to being implemented within the prediction system **100** but is instead one example of a system that may implement the method **500**.

[0048] At **510**, the prediction system **100** estimates a local vector of depth for an image by combining random noise with an image embedding and a geometric embedding using a learning model. The image embedding can have pixel data about the image that is sparse about objects within a view, a shot, etc. The geometric embedding can be camera intrinsics and camera extrinsics. As previously explained, the camera intrinsics can include a focal length, a principal point, a pixel aspect-ratio, a skew, etc., defining the geometries for a camera (e.g., a monocular camera, a RGB camera, etc.). The camera extrinsics can include parameters that describe a position, orientation, etc., of the camera relative to a reference frame.

[0049] Moreover, the local vector represents pixel-level depth through the learning model (e.g., a neural network) locally conditioning the image embedding with the geometric embedding. In one approach, conditioning involves controlling diffusion through adding a variable for sampling, such as a class label, a language caption, camera parameters, etc. Local conditioning prepares the image embedding and the geometric embedding for diffusion through adding depth information that is unavailable at certain pixels. Depth information may particularly be unavailable when the image embedding has sparse and unstructured data.

[0050] In one embodiment, the local conditioning concatenates corresponding tokens for the image embedding and the geometric embedding token-by-token. The prediction system **100** can simplify parsing by the learning model with forming embeddings representing vectors having projected information and dimensionality information. As previously explained, a token captures contextual significance of an input element (e.g., a pixel) within a sequence that is derived through weighted aggregation from information associated with other elements. In another approach, the prediction system **100** generates the geometric embedding to describe a pixel in a 3D reference frame rather than 2D location within the image. In this way, diffusion focuses upon localized predictions for a depth map that reduces ambiguity between camera geometries for an image token, thereby improving system robustness.

[0051] At **520**, the prediction system **100** predicts a global vector through combining the image embedding and the geometric embedding at a scene-level by the learning model. Here, the learning model globally conditions the image embedding and the geometric embedding to predict the global vector that incorporates spatial structure about the image. For example, image embeddings are generated using a convolutional encoder resulting in multi-scale feature maps at increasingly lower resolutions during global conditioning. To combine this multi-scale information, lower-resolution feature maps are up-sampled and concatenated for completeness. In this way, the image embeddings contain image data that is multi-resolution, multi-scale, and scene-level yet dense

within the global vector. This enhances global consistency independent of depth during diffusion for corresponding image embeddings token-by-token.

[0052] At **530**, the depth module **130** infers a depth map of the image by combining the local vector and the global vector using a diffusion model. In various implementations, the diffusion model is a RIN that processes the random noise, local tokens, and global tokens and outputs noise representing a depth map. As previously explained, diffusion of the local and global tokens continues with outputting noise until identifying a depth map through convergence. For example, RIN executes attention operations using latent tokens that attend to inputted local tokens and global tokens through a read operation. The RIN can include stacked RIN blocks for forming a deeper model that infers the depth map. Furthermore, the read operation is followed by several self-attention computations on outputs and the final write from latents to inputs.

[0053] In one approach, the prediction system **100** groups tokens representing pixels and scene level information within the image using the local and global tokens. For example, the grouped tokens are in a series having a group for the local tokens and a group for the global tokens that simplifies computations through organization. The grouped tokens can include sparse geometric information but exclude information about spatial structure for objects within a scene. Furthermore, the RIN can run several times using the grouped tokens for predicting and removing noise to identify the depth map. In one approach, the depth map develops as an accurate representation of depth at a pixel-level through diffusion computations that are efficient regardless of sparse and unstructured pixel data derived from the image. Accordingly, the prediction system implements diffusion using a RIN and locally and globally conditioned embeddings about an image from a lone camera, thereby improving depth map inferences.

[0054] Detailed embodiments are disclosed herein. However, it is to be understood that the disclosed embodiments are intended as examples. Therefore, specific structural and functional details disclosed herein are not to be interpreted as limiting, but merely as a basis for the claims and as a representative basis for teaching one skilled in the art to variously employ the aspects herein in virtually any appropriately detailed structure. Furthermore, the terms and phrases used herein are not intended to be limiting but rather to provide an understandable description of possible implementations. Various embodiments are shown in FIGS. **1-5**, but the embodiments are not limited to the illustrated structure or application.

[0055] The flowcharts and block diagrams in the figures illustrate the architecture, functionality, and operation of possible implementations of systems, methods, and computer program products according to various embodiments. In this regard, a block in the flowcharts or block diagrams may represent a module, segment, or portion of code, which comprises one or more executable instructions for implementing the specified logical function(s). It should also be noted that, in some alternative implementations, the functions noted in the block may occur out of the order noted in the figures. For example, two blocks shown in succession may, in fact, be executed substantially concurrently, or the blocks may sometimes be executed in the reverse order, depending upon the functionality involved.

[0056] The systems, components, and/or processes described above can be realized in hardware or a combination of hardware and software and can be realized in a centralized fashion in one processing system or in a distributed fashion where different elements are spread across several interconnected processing systems. Any kind of processing system or another apparatus adapted for carrying out the methods described herein is suited. A typical combination of hardware and software can be a processing system with computer-usable program code that, when being loaded and executed, controls the processing system such that it carries out the methods described herein.

[0057] The systems, components, and/or processes also can be embedded in a computer-readable storage, such as a computer program product or other data programs storage device, readable by a machine, tangibly embodying a program of instructions executable by the machine to perform methods and processes described herein. These elements also can be embedded in an application

product which comprises the features enabling the implementation of the methods described herein and, which when loaded in a processing system, is able to carry out these methods.

[0058] Furthermore, arrangements described herein may take the form of a computer program product embodied in one or more computer-readable media having computer-readable program code embodied, e.g., stored, thereon. Any combination of one or more computer-readable media may be utilized. The computer-readable medium may be a computer-readable signal medium or a computer-readable storage medium. The phrase “computer-readable storage medium” means a non-transitory storage medium. A computer-readable storage medium may be, for example, but not limited to, an electronic, magnetic, optical, electromagnetic, infrared, or semiconductor system, apparatus, or device, or any suitable combination of the foregoing. More specific examples (a non-exhaustive list) of the computer-readable storage medium would include the following: a portable computer diskette, a hard disk drive (HDD), a solid-state drive (SSD), a ROM, an EPROM or flash memory, a portable compact disc read-only memory (CD-ROM), a digital versatile disc (DVD), an optical storage device, a magnetic storage device, or any suitable combination of the foregoing. In the context of this document, a computer-readable storage medium may be any tangible medium that can contain, or store a program for use by or in connection with an instruction execution system, apparatus, or device.

[0059] Generally, modules as used herein include routines, programs, objects, components, data structures, and so on that perform particular tasks or implement particular data types. In further aspects, a memory generally stores the noted modules. The memory associated with a module may be a buffer or cache embedded within a processor, a RAM, a ROM, a flash memory, or another suitable electronic storage medium. In still further aspects, a module as envisioned by the present disclosure is implemented as an ASIC, a hardware component of a system on a chip (SoC), as a programmable logic array (PLA), or as another suitable hardware component that is embedded with a defined configuration set (e.g., instructions) for performing the disclosed functions.

[0060] Program code embodied on a computer-readable medium may be transmitted using any appropriate medium, including but not limited to wireless, wireline, optical fiber, cable, radio frequency (RF), etc., or any suitable combination of the foregoing. Computer program code for carrying out operations for aspects of the present arrangements may be written in any combination of one or more programming languages, including an object-oriented programming language such as Java™, Smalltalk™, C++, or the like and conventional procedural programming languages, such as the “C” programming language or similar programming languages. The program code may execute entirely on the user's computer, partly on the user's computer, as a stand-alone software package, partly on the user's computer and partly on a remote computer, or entirely on the remote computer or server. In the latter scenario, the remote computer may be connected to the user's computer through any type of network, including a local area network (LAN) or a wide area network (WAN), or the connection may be made to an external computer (for example, through the Internet using an Internet Service Provider).

[0061] The terms “a” and “an,” as used herein, are defined as one or more than one. The term “plurality,” as used herein, is defined as two or more than two. The term “another,” as used herein, is defined as at least a second or more. The terms “including” and/or “having,” as used herein, are defined as comprising (i.e., open language). The phrase “at least one of . . . and . . .” as used herein refers to and encompasses any and all combinations of one or more of the associated listed items. As an example, the phrase “at least one of A, B, and C” includes A, B, C, or any combination thereof (e.g., AB, AC, BC, or ABC).

[0062] Aspects herein can be embodied in other forms without departing from the spirit or essential attributes thereof. Accordingly, reference should be made to the following claims, rather than to the foregoing specification, as indicating the scope hereof.

Claims

1. A prediction system comprising: a memory storing instructions that, when executed by a processor, cause the processor to: estimate a local vector for an image by combining random noise, an image embedding about the image, and a geometric embedding using a learning model, the local vector having depth information at a pixel-level for the image; predict a global vector by combining the image embedding and the geometric embedding at a scene-level by the learning model; and infer a depth map of the image by combining the local vector and the global vector using a diffusion model.
2. The prediction system of claim 1 further including instructions to: estimate a depth noise by the diffusion model combining injected noise, latent information, and a ground-truth depth associated with picture data, wherein the diffusion model is a recurrent interface network (RIN) and the picture data has pixel data that is sparse; calculate a loss as a difference between the depth noise and the injected noise; and train the diffusion model with the loss.
3. The prediction system of claim 2 further including instructions to: remove an invalid subset of the pixel data by the diffusion model using dropout.
4. The prediction system of claim 1 further including instructions to: input the depth map as noise to the learning model; and alter the depth map by the diffusion model using local tokens at the pixel-level and global tokens, wherein the diffusion model is a recurrent interface network (RIN) having self-attention layers that iteratively process the local tokens and the global tokens.
5. The prediction system of claim 1, wherein the instructions to estimate the local vector further include instructions to: condition the image embedding and the geometric embedding locally at pixel positions for the depth information, wherein the image embedding and the geometric embedding are vectors having projected information and dimensionality information.
6. The prediction system of claim 1, wherein the instructions to predict the global vector further include instructions to: condition the image embedding and the geometric embedding globally independent from localized depth.
7. The prediction system of claim 1, wherein the instructions to infer the depth map further include instructions to: generate tokens representing the local vector and the global vector, wherein the tokens are pixels excluding information about spatial structure for objects within the image; and concatenate the tokens in a series having local tokens that are grouped and global tokens that are grouped.
8. The prediction system of claim 1, wherein the global vector has the image embedding and the geometric embedding at the scene-level across multiple scales of the image.
9. The prediction system of claim 1, wherein the diffusion model is a recurrent interface network (RIN) that operates within a dimensionality space that is reduced and the RIN factors camera extrinsics and camera intrinsics from the geometric embedding.
10. A non-transitory computer-readable medium comprising: instructions that when executed by a processor cause the processor to: estimate a local vector for an image by combining random noise, an image embedding about the image, and a geometric embedding using a learning model, the local vector having depth information at a pixel-level for the image; predict a global vector by combining the image embedding and the geometric embedding at a scene-level by the learning model; and infer a depth map of the image by combining the local vector and the global vector using a diffusion model.
11. The non-transitory computer-readable medium of claim 10 further including instructions to: estimate a depth noise by the diffusion model combining injected noise, latent information, and a ground-truth depth associated with picture data, wherein the diffusion model is a recurrent interface network (RIN) and the picture data has pixel data that is sparse; calculate a loss as a difference between the depth noise and the injected noise; and train the diffusion model with the loss.

- 12.** A method comprising: estimating a local vector for an image by combining random noise, an image embedding about the image, and a geometric embedding using a learning model, the local vector having depth information at a pixel-level for the image; predicting a global vector by combining the image embedding and the geometric embedding at a scene-level by the learning model; and inferring a depth map of the image by combining the local vector and the global vector using a diffusion model.
- 13.** The method of claim 12 further comprising: estimating a depth noise by the diffusion model combining injected noise, latent information, and a ground-truth depth associated with picture data, wherein the diffusion model is a recurrent interface network (RIN) and the picture data has pixel data that is sparse; calculating a loss as a difference between the depth noise and the injected noise; and training the diffusion model with the loss.
- 14.** The method of claim 13 further comprising: removing an invalid subset of the pixel data by the diffusion model using dropout.
- 15.** The method of claim 12 further comprising: inputting the depth map as noise to the learning model; and altering the depth map by the diffusion model using local tokens at the pixel-level and global tokens, wherein the diffusion model is a recurrent interface network (RIN) having self-attention layers that iteratively process the local tokens and the global tokens.
- 16.** The method of claim 12, wherein estimating the local vector further includes: conditioning the image embedding and the geometric embedding locally at pixel positions for the depth information, wherein the image embedding and the geometric embedding are vectors having projected information and dimensionality information.
- 17.** The method of claim 12, wherein predicting the global vector further includes: conditioning the image embedding and the geometric embedding globally independent from localized depth.
- 18.** The method of claim 12, wherein inferring the depth map further includes: generating tokens representing the local vector and the global vector, wherein the tokens are pixels excluding information about spatial structure for objects within the image; and concatenating the tokens in a series having local tokens that are grouped and global tokens that are grouped.
- 19.** The method of claim 12, wherein the global vector has the image embedding and the geometric embedding at the scene-level across multiple scales of the image.
- 20.** The method of claim 12, wherein the diffusion model is a recurrent interface network (RIN) that operates within a dimensionality space that is reduced and the RIN factors camera extrinsics and camera intrinsics from the geometric embedding.
-